High-Volume Insurance Tariff Evaluation

Programmable insurance mathematics • versioned execution graphs • independent numerical validation

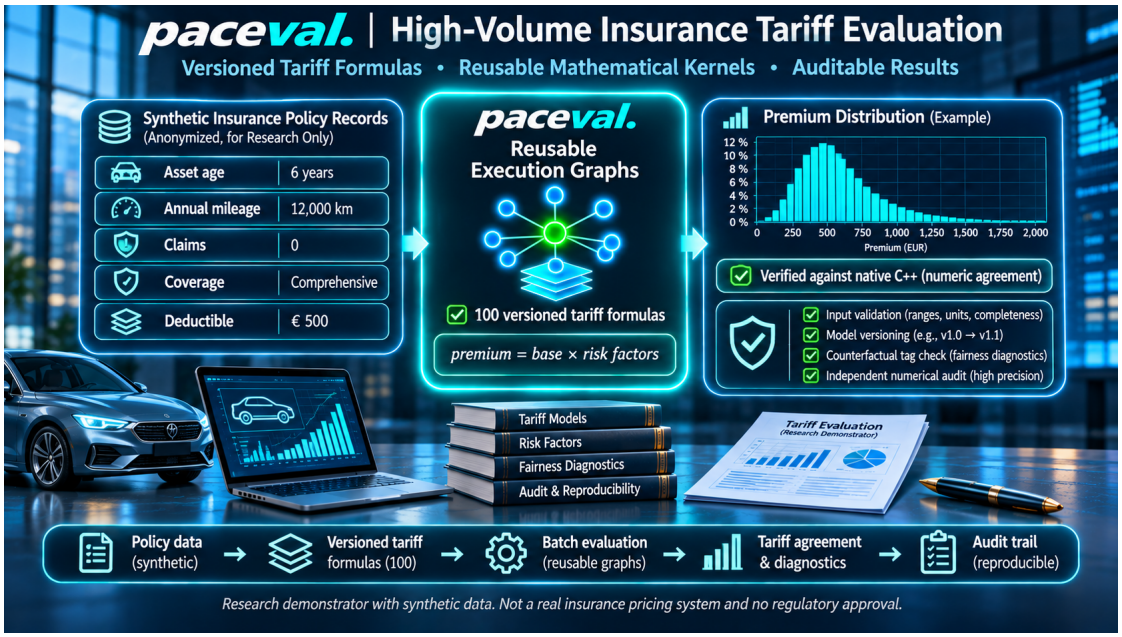


Figure 1. Illustrative research workflow. The actual measurements and software implementation are described on the following pages.

Verified result

The completed C++ study created 100 reusable mathematical models and performed 102,400 premium calculations on 1,024 synthetic policy records. All evaluations passed an independent 100-digit numerical audit at the specified tolerance. Every rating-vector pair was invariant to a deliberately excluded non-rating tag.

Technical scope: mathematical execution and reproducibility; not a legally approved insurance tariff, real customer quotation, or a general fairness certification.

1. Research question and regulatory context

Can a large collection of explicit synthetic insurance tariff formulas be represented as reusable paceval computation graphs, evaluated with changing policy inputs, updated by version, and independently audited without altering the meaning of the mathematical expressions?

The starting five-page proposal WP-013 illustrated premium = b × (1 + a × age) × (1 + c × claim_count). This implementation deliberately defines age as the age of the insured asset, not the age of the insured person. The demonstration extends that baseline to mileage, coverage and deductible and introduces an intentionally narrow paired non-rating tag diagnostic.

External research and supervisory setting

The Casualty Actuarial Society describes cost-estimation principles used in property and casualty ratemaking. EIOPA (2023) warns against unfair premium differentiation not linked to risk or service costs; its 2025 AI governance Opinion discusses explainability, documentation, fairness and human oversight. EU Test-Achats guidance addresses unisex individual insurance premium calculation. These requirements apply independently of how accurately a mathematical function is executed.

No insurer data, customer pricing, approval status, protected personal attributes or actuarially fitted claim frequencies are included. All observed numeric results come from an explicitly deterministic synthetic workload.

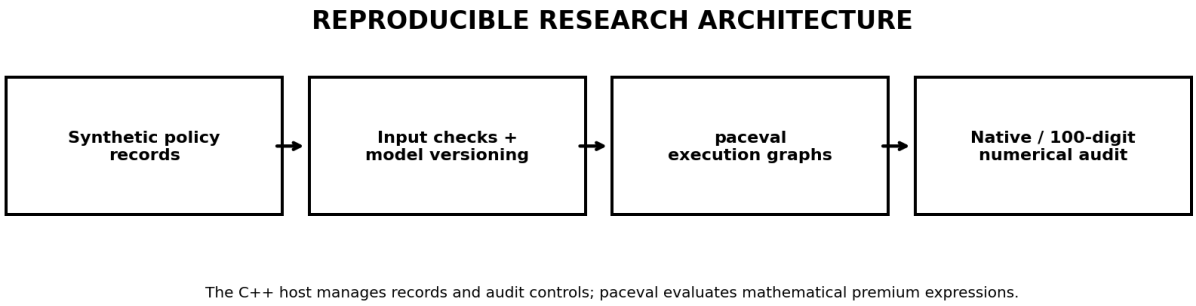


Figure 2. Data flow and responsibility boundary: C++ host on both sides of the paceval execution graph.

2. Mathematical tariff families

The five paceval input variables are age (insured asset years, 0–25), claims (integer 0–4), km (thousands of km annually, 1–50), cover (synthetic category 1–3), and excess (thousands of EUR, 0–2). All are bounded by input validation before any computation.

For coefficients b, a, c, m, g, h, f that vary by model ID, the four families are:

Family	Exact synthetic mathematical structure
F1	b × (1 + a·age) × (1 + c·claims)
F2	F1 × (1 + m·(km − 12)/12) × (1 + g·(cover − 1))
F3	b × (1 + a·age) × (1 + c·claims) × (1 + g·(cover − 1)) × exp(m·(km − 12)/12) / (1 + h·excess)
F4	b × [1 + a·age + c·claims + m·km/12 + g·(cover − 1) + f·age·claims] / (1 + h·excess)

All 100 explicit source expressions and coefficients are included in examples/models.csv. They are declared in the SDK only once, then reused over all policy inputs. The actual form of each graph is independent of the nonrating_tag field.

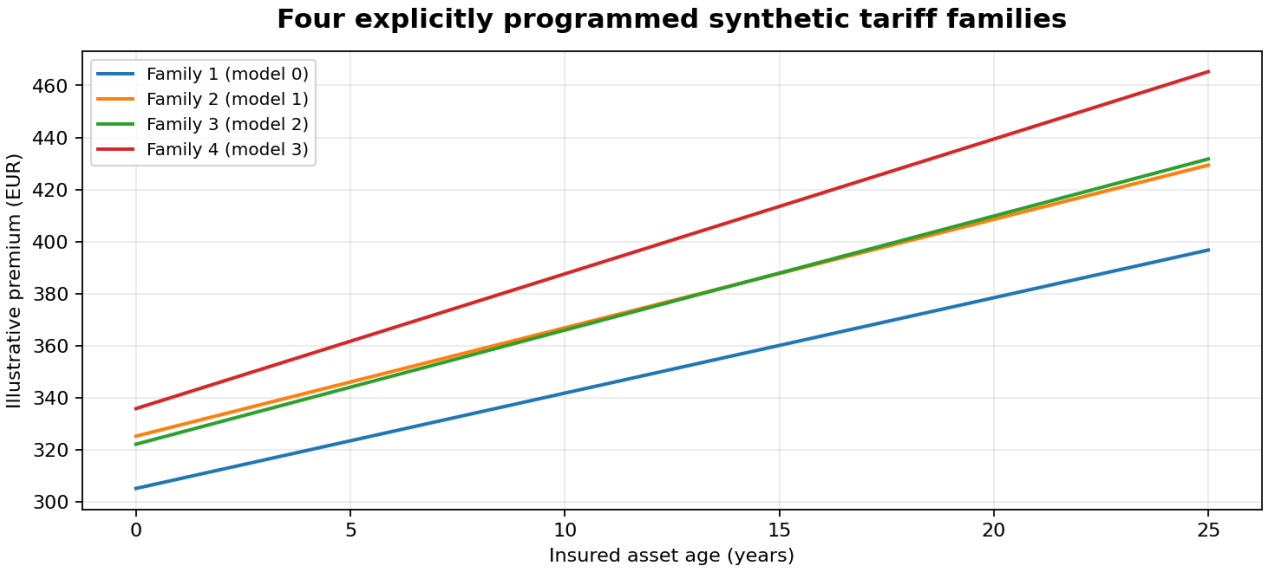


Figure 3. Selected models for one controlled input sweep. The curves represent illustrative rules, not market premium observations.

3. Direct paceval C++ integration

The C++17 application includes the supplied official paceval_main.h and compiles paceval_main.cpp directly. CMake links the separately supplied architecture-compatible libpaceval_linux_sharedLIB.so. No custom dynamic symbol loader is used. The published ZIP excludes the external runtime but preserves the original licensing comments in the SDK files.

paceval calls actually used: paceval_InitializeLibrary, paceval_CreateComputation, paceval_dGetComputationResultExt, paceval_dGetComputationResult, paceval_GetIsError, paceval_DeleteComputation and paceval_FreeLibrary. Computation handles are RAII-managed and destroyed before library shutdown.

For each formula, the host assembles fixed-size batches of five floating-point inputs. The graph is evaluated with the SDK batch API; its Boolean return is interpreted according to the SDK convention: true denotes an error. Each result is separately checked against a native C++ function implementing the same family.

Reproducibility and model governance

A fixed seed creates 512 synthetic rating vectors. Each is duplicated under non-rating tag 0 and 1, yielding 1,024 policy rows. A model record contains its ID, integer version, coefficients and full textual expression. Updating model 17 from v1 to v2 creates a new handle without invalidating model 18 or any unrelated graph.

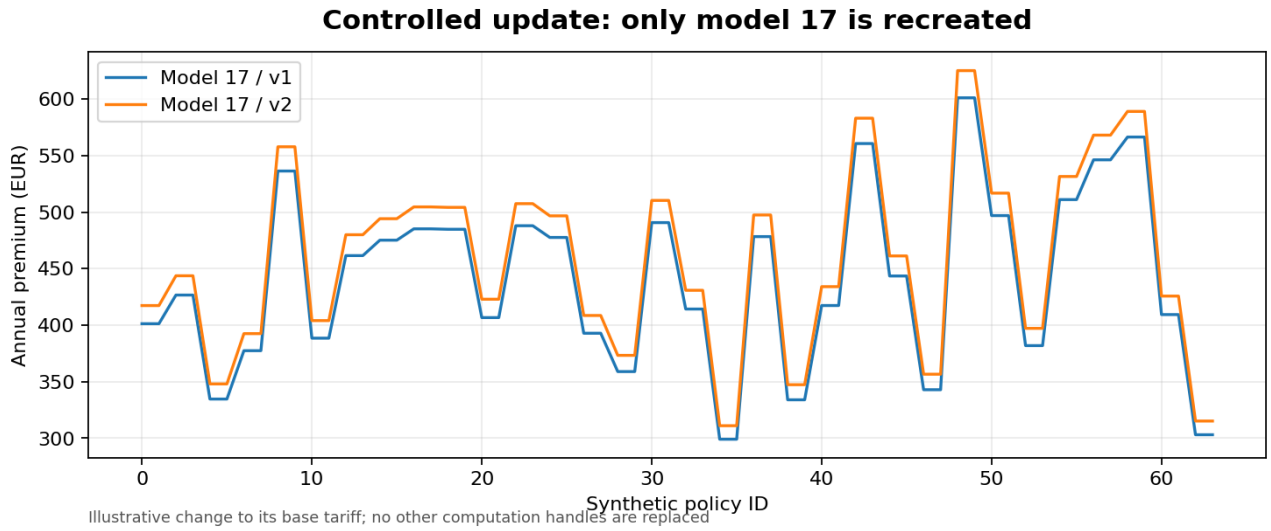


Figure 4. Reproducible controlled update, with both versions retained for audit.

4. Numerical validation and error bounds

Independent verification	Observed result
Actual paceval premium evaluations	102,400 across 100 models and 1,024 rows
Native C++ comparison	All within $2 \times 10^{-10} \times (1 + |\text{expected}|)$ absolute tolerance
100-digit independent arithmetic	102,400/102,400 within $10^{-10} \times (1 + |\text{reference}|)$
Maximum high-precision absolute discrepancy	4.197e-13 EUR
Counterfactual non-rating tag pair differences	All zero in these 512 synthetic pairs and 100 models
Smooth-control TINC raw intervals	24/24 contain independent references

The independent Python reference uses mpmath at 100 decimal digits, with every CSV numerical input first parsed as a binary64 value and then converted exactly to high precision. It computes the four tariff families separately from the paceval parser and C++ native reference. A strict assertion rejects any result outside the prescribed tolerance.

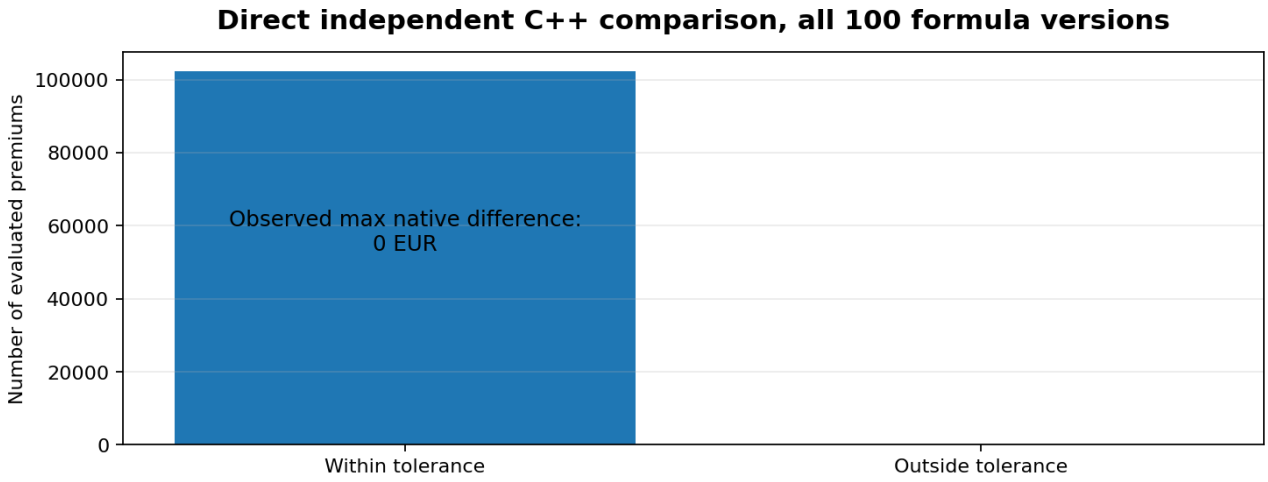


Figure 5. All recorded premium results satisfy the independent native comparison criterion.

5. Synthetic tariff portfolio examination

The synthetic records are not drawn from real claims data and do not contain protected personal attributes. Pricing values are meaningful only in the constructed example; their distribution is a software demonstration and cannot be interpreted as calibrated loss cost, customer fairness or commercial premium adequacy.

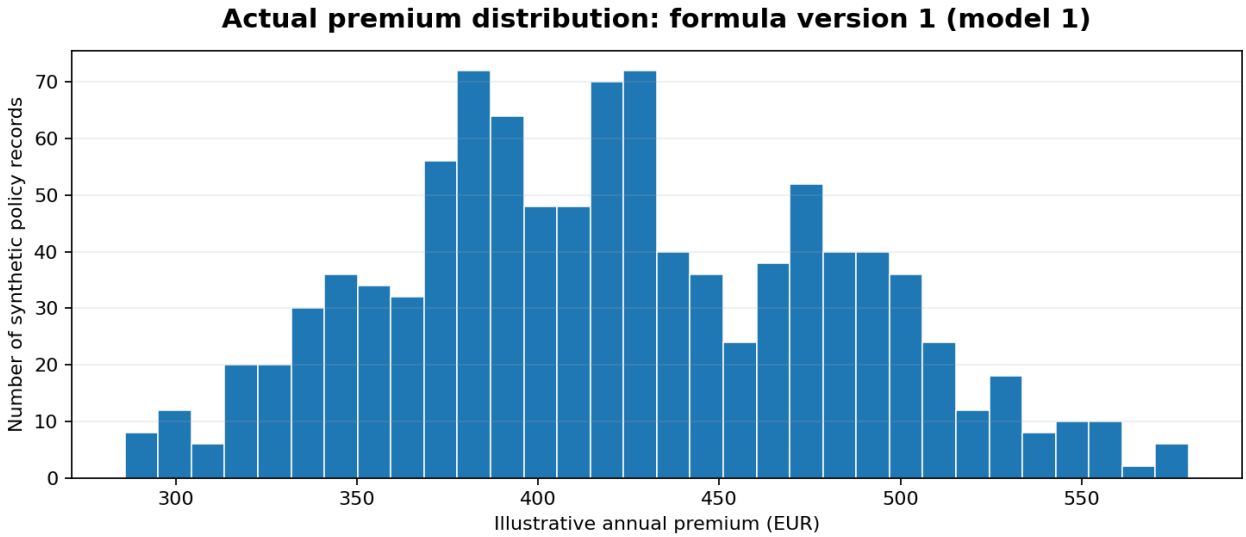


Figure 6. Actual output distribution for one formula over the exported synthetic policy records.

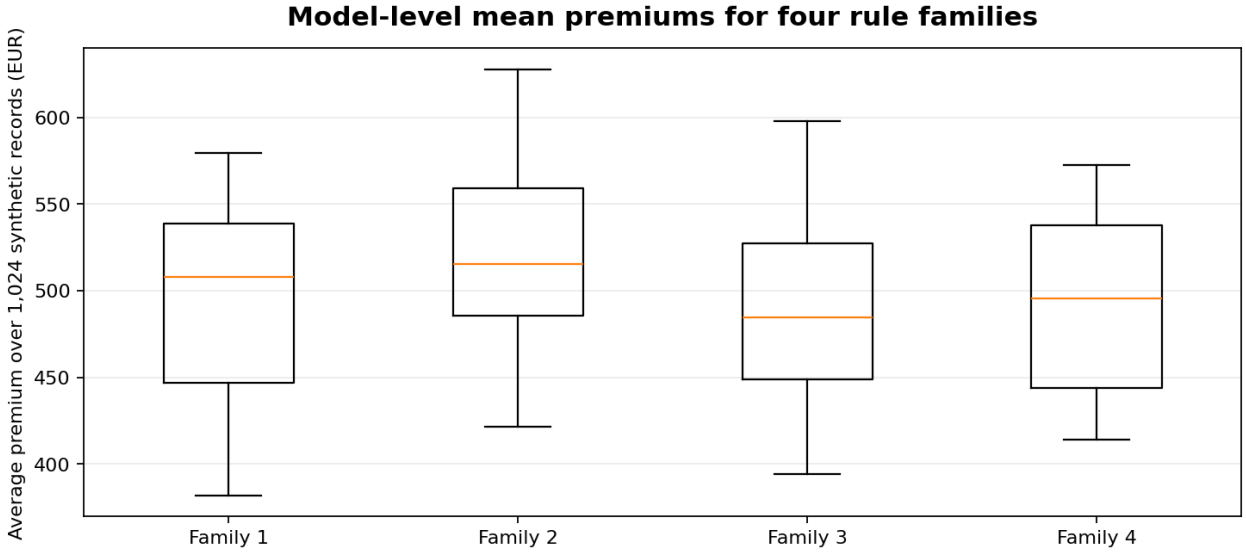


Figure 7. Differences among model family averages reflect chosen synthetic coefficients, not actual insurer risk estimates.

6. Input validation and bounded fairness diagnostics

The C++ host rejects nonfinite numeric inputs, asset ages outside 0–25, non-integer or out-of-range claims, mileage outside 1–50 thousand km/year, invalid coverage codes and deductibles outside 0–2 thousand EUR. Malformed CSV rows and broken tag pairs are also rejected.

For a controlled paired sensitivity audit, the demonstration creates two policy rows with identical five rating inputs and opposite nonrating_tag values. The tag is excluded from the paceval variable declaration and from all four expression families. Across all 100 models and 512 pairs, observed premium differences are exactly zero.

This result checks only explicit tag invariance of the implemented functions. It does not rule out indirect discrimination through proxies, assess price walking or certify fairness under EU law. Such assessments require appropriate substantive claims data, relevant protected-group analysis where lawful, and independent legal/actuarial governance.

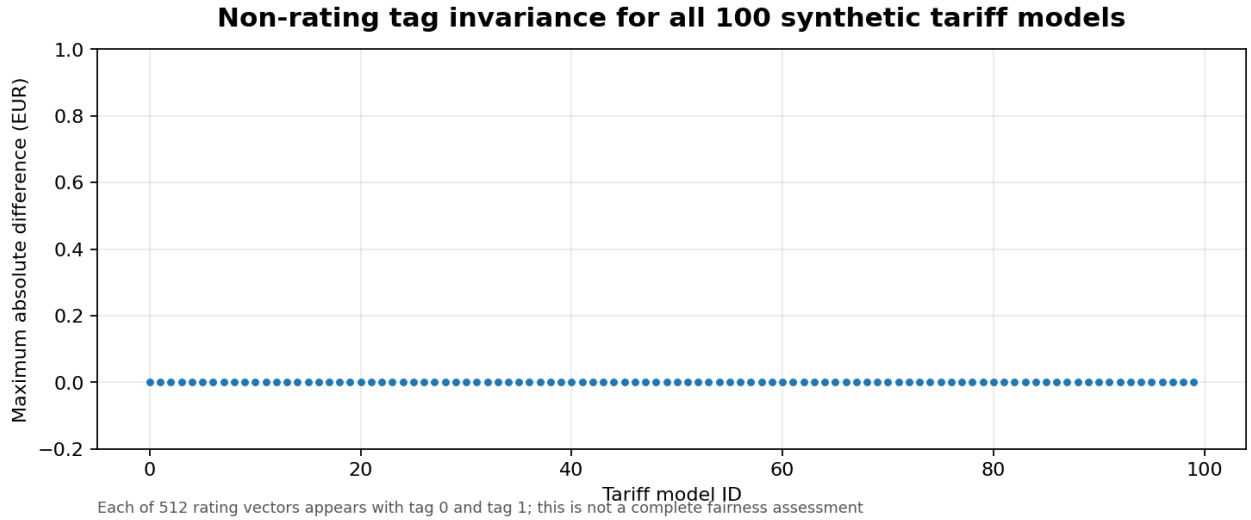


Figure 8. Observed invariance for deliberately controlled, equal-input synthetic pairs.

7. Separate TINC diagnostic and reproducibility

As an additional diagnostic, the project creates a separate TINC computation graph for the smooth polynomial age² + claims² and tests 24 points. Both input variables are exact small integers in these samples. Every unmodified raw interval contains the high-precision polynomial result. This control test is deliberately separate from the insurance tariff formulas; it is not a formal guarantee about arbitrary TINC results.

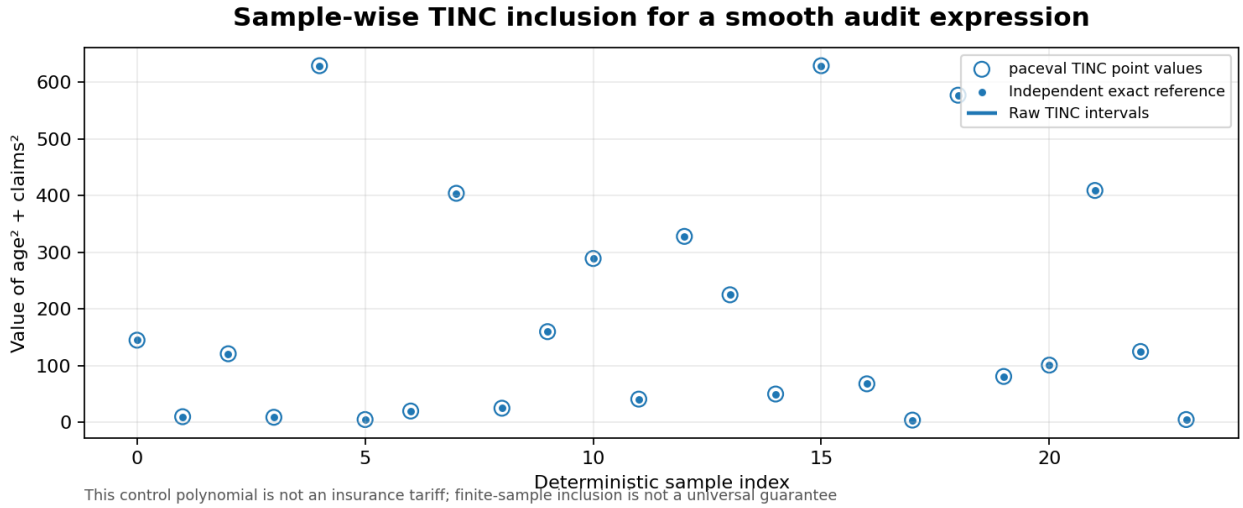


Figure 9. Raw TINC sample intervals and independently computed exact polynomial references.

How to reproduce

Build src/insurance_tariffs.cpp and src/paceval_main.cpp with the provided src/CMakeLists.txt and an external compatible Linux paceval runtime. Run insurance_tariffs with --models 100 --pairs 512 --batch 128 --seed 20261112 --out run, then run python3 src/high_precision_reference.py run. The repository includes source scripts, CSV results, a JSON summary and eight diagrams in PNG/SVG formats.

GCC and Clang builds, CTest validation, invalid-input rejection, batch invariance and a fresh ZIP extraction/rebuild are included in the release verification procedure.

8. References and boundaries

Source of the case: WP_013_High_Volume_Insurance_Tariff_Evaluation.pdf (user-supplied five-page scientific research proposal, November 2026). The present document extends its synthetic mathematical workload; it is not a measured performance study.

Official paceval C/C++ API: https://paceval.com/api/

CAS — P&C; Insurance Ratemaking Principles: https://www.casact.org/statement-principles-regarding-property-and-casualty-insurance-ratemaking

EIOPA — differential pricing supervisory statement, 2023: https://www.eiopa.europa.eu/publications/supervisory-statement-differential-pricing-practices-non-life-insurance-lines-business_en

EIOPA — Opinion on AI governance and risk management, 2025: https://www.eiopa.europa.eu/eiopa-publishes-opinion-ai-governance-and-risk-management-2025-08-06_en

EUR-Lex — Test-Achats/unisex insurance guidance: https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:52012XC0113(01)

Scientific and publication limits

The empirical result is successful mathematical computation of synthetic tariff expressions with reproducible numerical and versioning checks. The work provides no evidence that the sample pricing factors are actuarially adequate or legally permissible in a particular jurisdiction. It does not establish security of real customer data, absence of proxy discrimination, suitability of an underwriting decision or commercial performance benefits.

The source package contains commented C++ code, official SDK facade files, Python reference tests, deterministic synthetic datasets, and all cited local diagrams. No performance speedups or hardware acceleration claims are reported. No proprietary Linux runtime library or separate LICENSE_NOTICE.md is included.